

TP2 – Langage Compilés

Cyril MAZAURIC

Partie 1 -- Les 4 etapes de la compilation	1
1.2 Exercice : observer chaque etape.....	1
Questions :	2
Partie 2 -- GCC en pratique	2
2.2 Exercice : compiler proprement	3
Questions :	3
Partie 3 -- Variables, types et operateurs	3
3.3 Exercice : explorateur de types	3
Question :	4
3.5 Exercice : calculs.....	4
Partie 4 -- Entrees/sorties avec scanf.....	5
4.2 Exercice : fiche etudiant interactive.....	5
Partie 5 -- Les erreurs en C	6
5.2 Exercice : provoquer et lire des erreurs	6
Partie 6 -- Mini-projet : convertisseur universel	7

Partie 1 -- Les 4 etapes de la compilation

1.2 Exercice : observer chaque etape

Creez le fichier hello.c suivant :

```
#include <stdio.h>
#define MESSAGE "Bonjour YNOV !"
int main(void) {
    printf("%s\n", MESSAGE);
    return 0;
}
```

Executez chaque commande une par une et observez les fichiers generes :

```
# Etape 1 : Preprocesseur uniquement
gcc -E hello.c -o hello.i

# Etape 2 : Compilation vers assembleur
gcc -S hello.c -o hello.s

# Etape 3 : Assemblage vers fichier objet
gcc -c hello.c -o hello.o

# Etape 4 : Edition de liens (produit l'executable)
gcc hello.o -o hello

# Execution
./hello
```

```
● jesui@ANV15C:~/langage_compiles/jour2$ touch hello.c
● jesui@ANV15C:~/langage_compiles/jour2$ nano hello.c
● jesui@ANV15C:~/langage_compiles/jour2$ gcc -E hello.c -o hello.i
● jesui@ANV15C:~/langage_compiles/jour2$ gcc -S hello.c -o hello.s
● jesui@ANV15C:~/langage_compiles/jour2$ gcc -c hello.c -o hello.o
● jesui@ANV15C:~/langage_compiles/jour2$ gcc hello.o -o hello
● jesui@ANV15C:~/langage_compiles/jour2$ ./hello
Bonjour YNOV !
```

Questions :

1. Ouvrez hello.i : que remarquez-vous par rapport au fichier source original ? Ou est passé #define MESSAGE ? Combien de lignes fait le fichier ?

Le fichier hello.i fait 820 lignes, le #define MESSAGE est devenu une partie du reste du fichier je pense, parmi les 800 autres lignes

2. Ouvrez hello.s : reconnaissez-vous des instructions ? Qu'est-ce que l'assembleur ?

Non je ne reconnais pas d'instruction, l'assembleur c'est un langage de bas niveau, presque compréhensible par le processeur.

3. Exécutez file hello.o puis file hello : quelle est la différence entre ces deux fichiers ?

Le fichier hello.o est un fichier objet, c'est à dire qu'il a été compilé par l'assembleur pour en faire un fichier objet, tandis que hello est en réalité un .exe

4. Quelle commande unique permet de réaliser les 4 étapes en une seule fois ?
"gcc -E hello.c -o hello.i && gcc -S hello.c -o hello.s && gcc -c hello.c -o hello.o && gcc hello.o -o hello"

Partie 2 -- GCC en pratique

2.2 Exercice : compiler proprement

Créez "test_options.c" :

Questions :

1. Quel warning obtenez-vous ? Que signifie-t-il ?

```
test_options.c:4:9: warning: unused variable 'x' [-Wunused-variable]
    4 |     int x;
      |         ^
```

on obtient un warning qui dit qu'une variable n'est pas utilisée, en l'occurrence c'est x noté avec sa position dans le code.

2. Comment corriger le code pour éliminer le warning ?

On pourrait enlever la variable puisqu'elle n'est pas utilisée ou alors lui attribuer une valeur, comme 1, null ou undefined.

3. Pourquoi est-il important de traiter les warnings même si le programme compile ?

Ça évitera des potentielles erreurs plus tard en interagissant avec d'autres parties du code.

Partie 3 -- Variables, types et opérateurs

3.3 Exercice : explorateur de types

Créez types.c qui affiche la taille en octets de chaque type C en utilisant sizeof :

```
#include <stdio.h>
int main(void) {
    printf("=== Tailles des types C ===\n");
    printf("char : %zu octet(s)\n", sizeof(char));
    printf("short : %zu octet(s)\n", sizeof(short));
    printf("int : %zu octet(s)\n", sizeof(int));
    printf("long : %zu octet(s)\n", sizeof(long));
    printf("float : %zu octet(s)\n", sizeof(float));
    printf("double : %zu octet(s)\n", sizeof(double));
    return 0;
}
```

Question :

Les tailles correspondent-elles au tableau ci-dessus ? Pourquoi peuvent-elles varier d'une machine à l'autre ?

Oui elles correspondent, la variation peut se faire en fonction de l'OS de la machine.

3.5 Exercice : calculs

Creez calculs.c qui :

1. Declare deux entiers a = 17 et b = 5
2. Affiche leur somme, difference, produit, quotient entier et reste (modulo)
3. Affiche le quotient en flottant (avec un cast)

Sortie attendue :

```
a = 17, b = 5
Somme      : 22
Difference  : 12
Produit     : 85
Quotient    : 3 (entier), 3.40 (flottant)
Reste       : 2
```

```
#include <stdio.h>
int main(void) {
    int a = 17;
    int b = 5;
    int somme = a + b;
    int difference = a - b;
    int produit = a * b;
    float quotient = (float)a / b;
    int reste = a % b;
    printf("a = 17, b = 5\n");
    printf("Somme      : %d\n", somme);
    printf("Différence : %d\n", difference);
    printf("Produit     : %d\n", produit);
    printf("Quotient    : %.2f\n", quotient);
    printf("Reste       : %d\n", reste);
    return 0;
}
```

```

● jesui@ANV15C:~/langage_compiles/jour2$ gcc calculs.c -o calculs
● jesui@ANV15C:~/langage_compiles/jour2$ ./calculs
a = 17, b = 5
Somme      : 22
Différence : 12
Produit    : 85
Quotient   : 3.40
Reste      : 2
jesui@ANV15C:~/langage_compiles/jour2$

```

Partie 4 -- Entrees/sorties avec scanf

4.2 Exercice : fiche etudiant interactive

Creez `fiche_interactive.c` qui demande a l'utilisateur :

- Son prenom
- Son age
- Sa note moyenne (float)

Puis affiche un resume formate.

```

#include <stdio.h>
int main(void) {
    int age;
    float taille;
    char prenom[30];
    printf("Quel est ton prenom ? ");
    scanf("%29s", prenom);
    printf("Quel age as-tu ? ");
    scanf("%d", &age);
    printf("Quelle taille (en m) ? ");
    scanf("%f", &taille);
    printf("Bonjour %s, %d ans, %.2f m.\n", prenom, age, taille);
    return 0;
}
● jesui@ANV15C:~/langage_compiles/jour2$ gcc fiche_interactive.c -o fiche
● jesui@ANV15C:~/langage_compiles/jour2$ ./fiche
Quel est ton prenom ? cyril
Quel age as-tu ? 19
Quelle taille (en m) ? 1.75
Bonjour cyril, 19 ans, 1.75 m.
jesui@ANV15C:~/langage_compiles/jour2$

```

Partie 5 -- Les erreurs en C

5.2 Exercice : provoquer et lire des erreurs

Creez un fichier erreurs.c et provoquez intentionnellement les 3 erreurs suivantes, une par une. Pour chaque erreur, copiez le message de GCC et expliquez-le :

1. Oubli du point-virgule : supprimez un ;

```
jesui@ANV15C:~/langage_compiles/jour2$ gcc erreurs.c -o erreur
erreurs.c: In function 'main':
erreurs.c:3:27: error: expected ';' before 'return'
   3 |     printf("cyril\n19 ans\n")
     |                               ^
     |                               ;
   4 |     return 10;
     |     ~~~~~
jesui@ANV15C:~/langage_compiles/jour2$
```

```
#include <stdio.h>
int main(void) {
    printf("cyril\n19 ans\n")
    return 10;
}
```

2. Type incompatible : affectez une chaine a un int (int x = "hello";)

```
jesui@ANV15C:~/langage_compiles/jour2$ gcc erreurs.c -o erreur
erreurs.c: In function 'main':
erreurs.c:3:10: warning: initialization of 'int' from 'char *' makes integer from pointer without a cast [-Wint-conversion]
   3 |     int x = "azy";
     |           ^~~~~
jesui@ANV15C:~/langage_compiles/jour2$
```

```
#include <stdio.h>
int main(void) {
    int x = "azy";
    return 0;
}
```

3. Fonction non declaree : appelez une fonction qui n'existe pas (maFonction();)

```

jesui@ANV15C:~/langage_compiles/jour2$ gcc erreurs.c -o erreur
erreurs.c: In function 'main':
erreurs.c:3:2: warning: implicit declaration of function 'maFonction' [-Wimplicit-function-declaration]
  3 |   maFonction();
    |   ~~~~~
/usr/bin/ld: /tmp/ccQTysvS.o: in function `main':
erreurs.c:(.text+0xe): undefined reference to `maFonction'
collect2: error: ld returned 1 exit status
jesui@ANV15C:~/langage_compiles/jour2$

```

```

#include <stdio.h>
int main(void) {
    maFonction();
    return 0;
}

```

Partie 6 -- Mini-projet : convertisseur universel

La calculatrice a déjà été vue en cours

place à un nouveau défi ! Créez `convertisseur.c` : un programme qui :

1. Affiche un menu avec 4 options de conversion :
 - Kilomètres → Miles
 - Kilogrammes → Livres (pounds)
 - Degrés Celsius → Fahrenheit
 - Quitter
2. Demande à l'utilisateur de choisir une option (int)
3. Demande la valeur à convertir (double)
4. Affiche le résultat avec 2 décimales
5. Gère les choix invalides (affiche une erreur et retourne 1)

Voici le code :

J'ai décidé de l'améliorer un peu en ajoutant :

- Un switch case à la place d'une suite de "if" et des "else", ce qui permet de gérer plus efficacement les différentes valeurs de scanf ainsi que le cas par défaut.
- Une fonction séparée pour afficher le menu de conversion afin de boucler sur le menu à la fin d'une conversion pour éviter les sorties involontaires.


```

#include <stdio.h>

int affichage_menu(void) {
    int option;
    printf("\n=== Convertisseur de mesures ===\n");
    printf("*-===== MENU *-=====*\n");
    printf("1. Kilomètres <-> Miles\n");
    printf("2. Kilogrammes <-> Livres\n");
    printf("3. Celsius <-> Fahrenheit\n");
    printf("4. Quitter\n");
    printf("Choisissez une option (1-4) : ");
    scanf("%d", &option);
    return option;
}

int main(void) {
    int Kilomètres = 0;
    float Miles = 0;
    int Kilogrammes = 0;
    float Livres = 0;
    int Celsius = 0;
    float Fahrenheit = 0;
    int option;

    while (1) {
        option = affichage_menu();
        switch (option) {
            case 1:
                printf("Entrez une distance en kilomètres : ");
                scanf("%d", &Kilomètres);
                Miles = Kilomètres * 0.621371;
                printf("%d km = %.2f miles\n", Kilomètres, Miles);
                break;
            case 2:
                printf("Entrez un poids en kilogrammes : ");
                scanf("%d", &Kilogrammes);
                Livres = Kilogrammes * 2.20462;
                printf("%d kg = %.2f livres\n", Kilogrammes, Livres);
                break;
            case 3:
                printf("Entrez une température en Celsius : ");
                scanf("%d", &Celsius);
                Fahrenheit = (Celsius * 9 / 5) + 32;
                printf("%d °C = %.2f °F\n", Celsius, Fahrenheit);
                break;
        }
    }
}

```

```

        case 4:
            printf("Au revoir !\n");
            return 0;
        default:
            printf("Option invalide. Veuillez choisir entre 1 et 4.\n");
        }
    }
    return 0;
}

```

jesui@ANV15C:~/langage_compiles/jour2\$ gcc convertisseur.c -o convertisseur
 jesui@ANV15C:~/langage_compiles/jour2\$./convertisseur

```

=== Convertisseur de mesures ===
*-----* MENU *-----*
1. Kilomètres <-> Miles
2. Kilogrammes <-> Livres
3. Celsius <-> Fahrenheit
4. Quitter
Choisissez une option (1-4) : 2
Entrez un poids en kilogrammes : 25
25 kg = 55.12 livres
  
```

```

=== Convertisseur de mesures ===
*-----* MENU *-----*
1. Kilomètres <-> Miles
2. Kilogrammes <-> Livres
3. Celsius <-> Fahrenheit
4. Quitter
Choisissez une option (1-4) : █
  
```